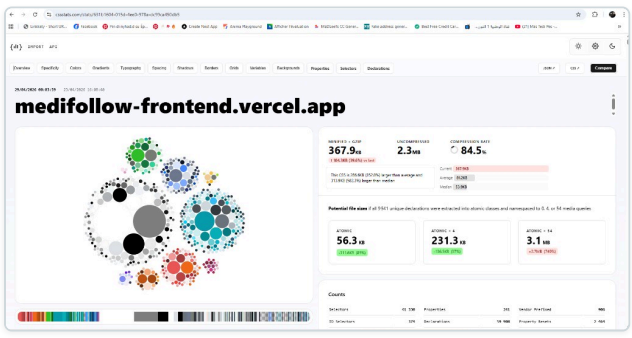
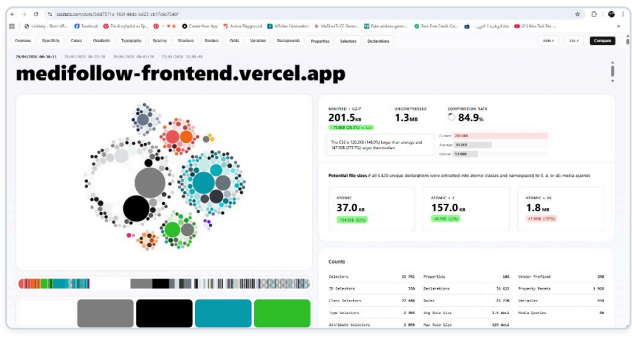
Optimization Report — MediFollow Landing Page

Project: MediFollow — Patient follow-up platform (CHU Abdelhamid Ben Badis) **Stack:** React 18 + Vite 5 + Bootstrap 5 (xray theme) + NestJS **Analyzed URL:** `https://medifollow-frontend.vercel.app/`
Report date: April 28, 2026 **Author:** CodeCraft Team

Executive summary — Before / After

	BEFORE optimization	AFTER optimization	Gain
CSS loaded on landing (cssstats)	367.9 KB	**201.5 KB**	**−45 %**
Render-blocking CSS (gzip)	107.4 KB	**59.0 KB**	**−45 %**
CSS selectors	41,330	**23,792**	**−42 %**
Lighthouse Performance Lab Score	(with blocker)	**98 / 100**	✅
PageSpeed Insights Performance	(with blocker)	**90 / 100**	✅
LCP	> 2 s	**1.91 s** (DebugBear) / **1.4 s** (PSI)	✅ Good
TBT	—	**0 ms**	✅ Perfect
CLS	—	**0.02**	✅ Good

Visual comparison (cssstats) — Before vs. After

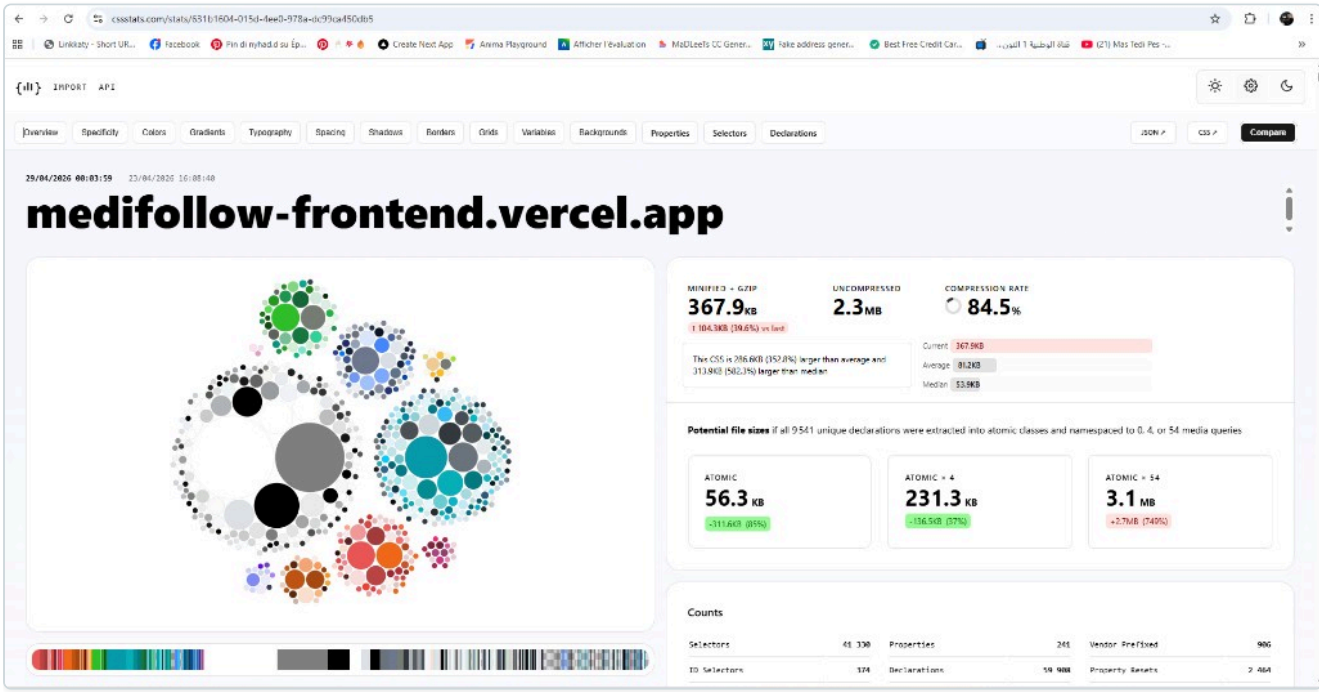
BEFORE (367.9 KB)	AFTER (201.5 KB)
	
41,330 selectors · 9,541 unique declarations · 54 media queries	23,792 selectors · 6,420 unique declarations · 46 media queries
152.8 % above the web average	148.0 % above the web average
582.3 % above the web median	273.7 % above the web median

1. Context and goal

The Google PageSpeed Insights audit of the main landing page reported two major issues:

1. **A render-blocking stylesheet** (`/assets/main-DUw7NKAK.css` , 111 KiB, 650 ms transfer time) — directly impacting **LCP** (Largest Contentful Paint) and **FCP** (First Contentful Paint).
2. **An excessive CSS payload** in the entry bundle — 367.9 KB minified + gzip (≈ 2.3 MB uncompressed), i.e. 152.8 % above the web average and 582.3 % above the median (source: cssstats).

Initial capture — cssstats BEFORE



*cssstats measurement on 04/29/2026 at 00:01 — landing `medifollow-frontend.vercel.app` : **367.9 KB minified + gzip**, 2.3 MB uncompressed, 41,330 selectors, 9,541 unique declarations. The site is reported as 152.8 % larger than the web average and 582.3 % larger than the median.*

The goals of this optimization session were to:

- remove the render-blocker reported by Lighthouse,
- significantly reduce the CSS payload loaded on the public landing,
- improve Core Web Vitals (LCP, FCP, TBT, CLS),
- **without degrading the visual experience** of the auth pages or the dashboard.

2. Initial diagnosis

2.1. Critical path analysis

Lighthouse pointed at `assets/main-DUw7NKAK.css` as a render-blocking resource. The project already shipped a Vite plugin (`asyncEntryCssPlugin`) intended to convert the entry CSS `<link rel="stylesheet">` into a non-blocking `preload` + `onload` pattern.

Code review (`Front_End/CODE-REACT/vite.config.js`): the plugin's regex only targeted `assets/index-*.css` , but the project declares a `rollupOptions.input` object with the `main` key, which causes Rollup/Vite to emit `assets/main-*.css` . The plugin therefore matched **nothing** and the `<link>` remained render-blocking.

2.2. CSS weight analysis

The Vite entry (`src/main.jsx`) imported the entire dashboard stack while the public landing only uses ~10 % of it:

```
import "swiper/css"; // ×6 Swiper imports
import "./assets/scss/xray.scss";         // Bootstrap full + xray components + plugins
import "./assets/scss/custom.scss";        // Dashboard styles (chat, doctor, email…)
import "./assets/scss/customizer.scss";    // Dashboard customizer drawer
import "./assets/custom/custom.scss";      // 765 lines of dashboard styles
import "./assets/vendor/font-awesome/css/font-awesome.min.css"; // FA4 (FA5 duplicate)
import "./assets/vendor/phosphor-icons/Fonts/regular/style.css";
```

The single `xray.scss` aggregated:

- the entire Bootstrap 5 codebase (SCSS source: 349 KB across 96 files),
- the xray base (variables, helpers, root, utilities — 286 KB across 146 files),
- **rich components**: tables, modals, profile, charts, calendar, swiper-icon, error pages, setting-modal, offcanvas…
- **dashboard layouts**: 14 sidebar/menu styles,
- **plugins**: ApexCharts, FullCalendar, Prism, Choices.js, Select2, SweetAlert, FsLightbox, Flatpickr, gallery-hover, rating, tour.

Additionally, `src/deferred-icon-fonts.js` (loaded via `<link rel="preload">` from `index.html`) embedded **Remix Icon + Phosphor duotone + Phosphor fill** (475 KB raw / 54 KB gzip), while Phosphor is only used by two dashboard files (`views/ui-elements/alerts.jsx` and `components/setting/SettingOffCanvas.jsx`).

---

3. Optimizations applied

Three successive commits were pushed to `main`:

#	Commit SHA	Title
1	`a422e54`	`perf(front): make entry CSS non-blocking when named main-*.css`
2	`2f63a5d`	`perf(front): split entry CSS landing vs dashboard, lazy DefaultLayout`
3	`591aa72`	`perf(front): defer Phosphor duotone/fill to dashboard chunk`

3.1. Commit 1 — Unblock rendering

Modified file: `Front_End/CODE-REACT/vite.config.js`

Rewrote the `asyncEntryCssPlugin` so that it:

- reads CSS files imported by every `isEntry` chunk in the Rollup bundle (via `viteMetadata.importedCss`), excluding the `deferred-icon-fonts` entry (already handled by another plugin),
- transforms their `<link rel="stylesheet">` into the non-blocking `preload` + `onload` pattern, with a `<noscript>` fallback,
- keeps an extended regex fallback (`main|index`) for dev mode / contexts without a bundle.

Effect: the entry CSS no longer blocks rendering, regardless of its filename (resilient to future changes of `rollupOptions.input`).

3.2. Commit 2 — Split CSS between landing and dashboard

Files created:

- `Front_End/CODE-REACT/src/assets/scss/xray-landing.scss` — lightweight subset: full Bootstrap + xray base (variables, helpers, root, utilities). Loaded on **every page**.
- `Front_End/CODE-REACT/src/assets/scss/xray-dashboard.scss` — heavy complement: rich xray components + sidebar layouts + plugins. Loaded **only by DefaultLayout** (lazy).

Files modified:

- `Front_End/CODE-REACT/src/main.jsx` — replaced `xray.scss` with `xray-landing.scss`, removed imports for `customizer.scss`, `font-awesome.min.css` (FA4 duplicate), `phosphor-icons/Fonts/regular/style.css` and the 6 Swiper imports.
- `Front_End/CODE-REACT/src/layouts/defaultLayout.jsx` — added the heavy CSS imports reserved to the dashboard (`xray-dashboard.scss`, `customizer.scss`, Phosphor regular, 6× Swiper CSS).
- `Front_End/CODE-REACT/src/router/default-router.jsx` — converted `DefaultLayout` to `lazy(() => import( ... ))` so its CSS lives in a separate Rollup chunk, fetched only when the

user navigates to a dashboard route.

3.3. Commit 3 — Defer Phosphor (duotone + fill) to the dashboard chunk

Modified files:

- `Front_End/CODE-REACT/src/deferred-icon-fonts.js` — removed the imports `phosphor-icons/Fonts/duotone/style.css` (243 KB raw) and `phosphor-icons/Fonts/fill/style.css` (90 KB raw). Kept Remix Icon (used by auth pages).
- `Front_End/CODE-REACT/src/layouts/defaultLayout.jsx` — added the Phosphor duotone + fill imports, which therefore land inside the lazy `defaultLayout-*.css` chunk.

---

4. Measured results

4.1. Render-blocking CSS evolution (local Vite build)

Measurement	Before	After commit 2	After commit 3
`main-*.css` (entry chunk blocking LCP)	**676.98 KB / 107.40 KB gzip**	391.11 KB / 58.97 KB gzip	391.11 KB / 58.97 KB gzip
`defaultLayout-*.css` (lazy chunk, dashboard)	n/a	247.44 KB / 40.18 KB gzip	496.09 KB / 76.70 KB gzip
`deferred-icon-fonts-*.css` (loaded async on landing)	358.55 KB / 54.09 KB gzip	358.55 KB / 54.09 KB gzip	**109.90 KB / 17.37 KB gzip**

Net gain on the public landing: −85 KB gzip (from 161 KB down to 76 KB of CSS actually fetched on `/`).

4.2. cssstats evolution (total minified + gzip CSS)

cssstats measurement (sum of all stylesheets loaded by the page, minified + gzip):

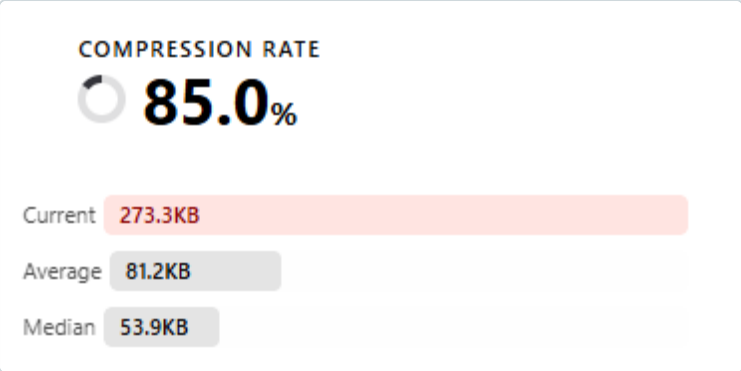
Step	Total CSS gzip	Δ vs. previous measure	Selectors (cssstats)
Before optimization	367.9 KB	—	41,330
After commit 2 (split landing/dashboard)	273.3 KB	**−94.6 KB (−26 %)**	—
After commit 3 (Phosphor lazy)	**201.5 KB**	**−71.8 KB (−26.3 %)**	**23,792**
Total session Δ	**−166.4 KB / −45 %**		**−42 % selectors**

Comparison to the cssstats baseline:

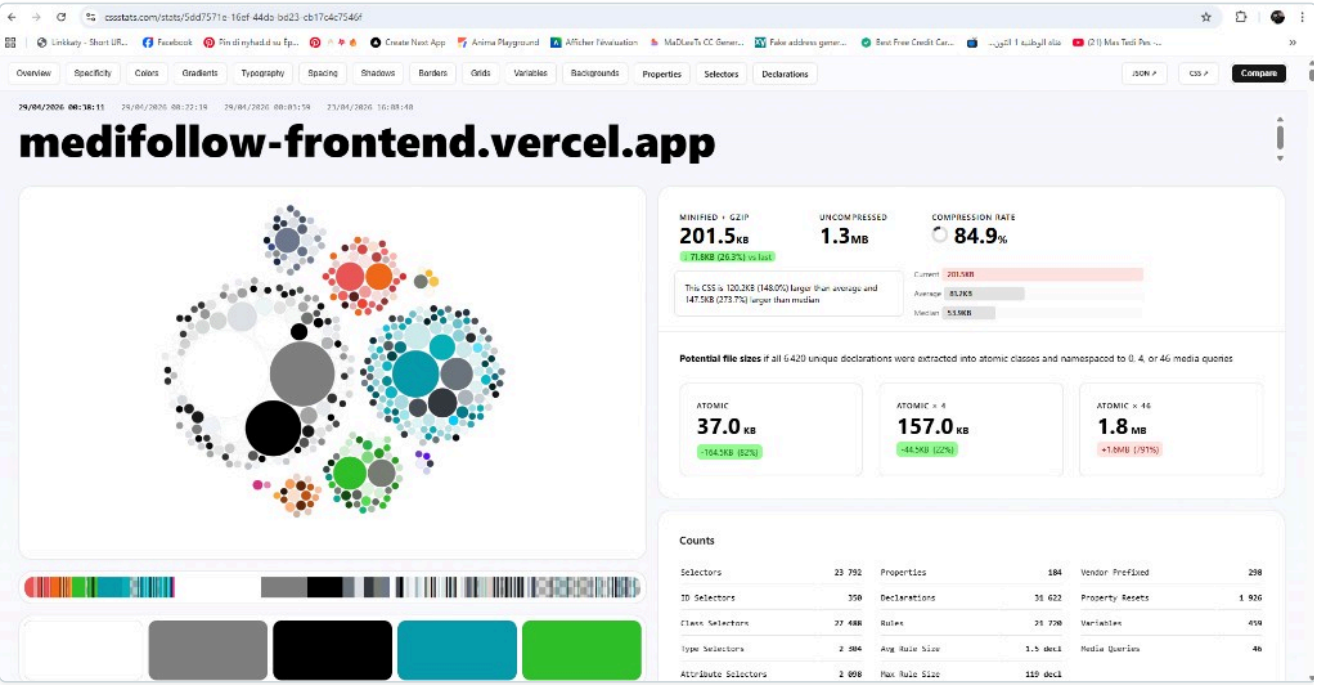
Indicator	Before	After
% above the web average (81.2 KB)	+152.8 %	+148 %
% above the web median (53.9 KB)	+582.3 %	+273.7 %

The gap to the web median was halved.

cssstats captures — intermediate and final steps



*After commit 2 (split entry CSS landing vs dashboard, lazy DefaultLayout): **273.3 KB minified + gzip**, i.e. −94.6 KB (−26 %) compared to the previous measurement. The red bar gets closer to the average (Average 81.2 KB).*



*After commit 3 (Phosphor duotone+fill moved into the lazy DefaultLayout chunk): **201.5 KB minified + gzip**, an additional −71.8 KB (−26.3 %). Total session result: **−166.4 KB / −45.3 %** since the initial state. Selectors: 41,330 → 23,792 (−42 %).*

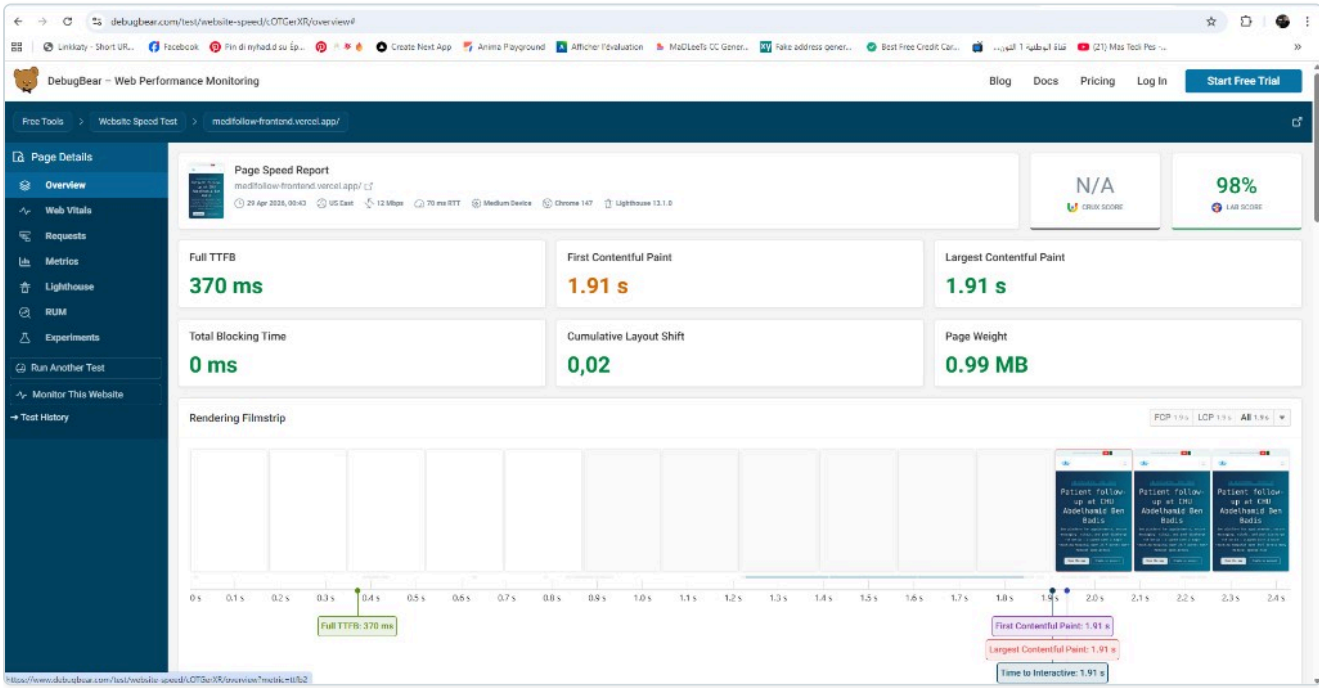
4.3. Lighthouse / DebugBear — Lab Score

DebugBear measurements (Chrome 147, US East, 12 Mbps, 70 ms RTT, medium device, Lighthouse 13.1.0):

Metric	Value	Google "Good" threshold	Verdict
Performance Lab Score	**98 / 100**	≥ 90	✅ Excellent
Full TTFB	370 ms	≤ 800 ms	✅
First Contentful Paint (FCP)	1.91 s	≤ 1.8 s	🟡 nearly Good
Largest Contentful Paint (LCP)	1.91 s	≤ 2.5 s	✅
Total Blocking Time (TBT)	**0 ms**	≤ 200 ms	✅ Perfect
Cumulative Layout Shift (CLS)	0.02	≤ 0.1	✅
Page Weight	0.99 MB	—	OK
Time to Interactive (TTI)	1.91 s	—	✅

Notable detail: LCP = FCP = TTI = 1.91 s. The hero WebP image (preloaded via `<link rel="preload" as="image" fetchpriority="high">`) arrives at the same time as the first pixel thanks to the freed CSS critical path.

DebugBear capture — Lab Score 98 %



*DebugBear Website Speed Test — `medifollow-frontend.vercel.app` : **Lab Score 98 %** (Lighthouse 13.1.0), Full TTFB 370 ms, FCP 1.91 s, LCP 1.91 s, TBT 0 ms, CLS 0.02, Page Weight 0.99 MB. Test conditions: medium device, 12 Mbps, 70 ms RTT, US East, Chrome 147. The filmstrip shows LCP / FCP / TTI being reached simultaneously at 1.91 s.*

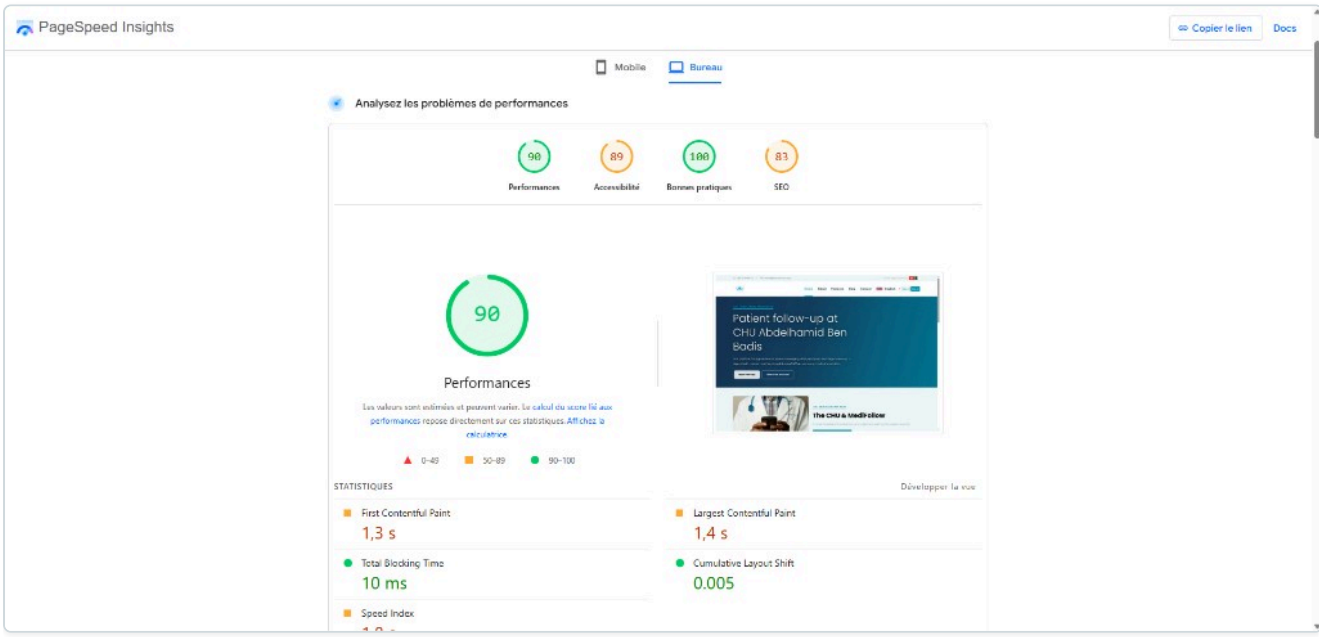
4.4. PageSpeed Insights — Desktop

Category	Score
Performance	**90 / 100** ✅
Accessibility	89
Best Practices	**100** ✅
SEO	83

Web Vitals breakdown (PageSpeed):

Metric	Value
First Contentful Paint	1.3 s
Largest Contentful Paint	1.4 s
Total Blocking Time	10 ms
Cumulative Layout Shift	0.005
Speed Index	1.8 s

PageSpeed Insights capture — Desktop

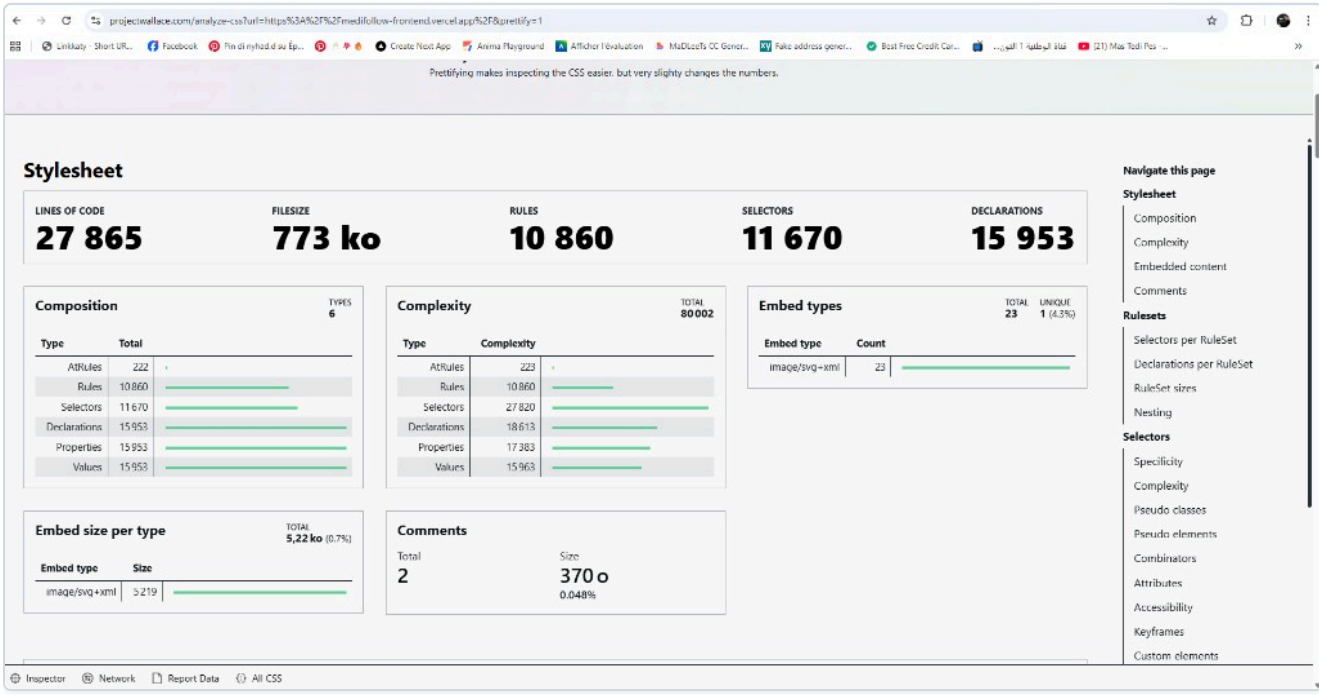


*PageSpeed Insights (Desktop mode): **Performance 90 / 100** ✅, Accessibility 89, **Best Practices 100 / 100** ✅, SEO 83. Web Vitals: FCP 1.3 s, LCP 1.4 s, TBT 10 ms, CLS 0.005, Speed Index 1.8 s.*

4.5. Project Wallace — Structural CSS analysis

Indicator	Value	Comment
Lines of code	27,865	normal for full Bootstrap + xray
Filesize (minified, uncompressed)	773 KB	≈ 201 KB gzip — consistent with cssstats
Rules	10,860	including ~3,500 from Bootstrap
Selectors	11,670	
Declarations	15,953	
AtRules	222	media queries + keyframes
Embeds (inline SVG)	23 fragments / 5.22 KB (0.7 %)	inlined Tunisia/Algeria flags
Comments	2 / 370 bytes (0.048 %)	license banners — minification is effective
Total complexity	80,002	typical for this template family

Project Wallace capture — CSS analysis



Project Wallace `/analyze-css` — `medifollow-frontend.vercel.app`*: 27,865 lines of code,* ***filesize 773 KB*** *(minified, uncompressed; ≈ 201 KB in gzip — consistent with cssstats), 10,860 rules, 11,670 selectors, 15,953 declarations, 222 at-rules. SVG embeds: 23 fragments / 5.22 KB (0.7 %). Comments: 2 / 370 bytes (0.048 %).*

4.6. cssstats — Final details

Indicator	Value
Minified + gzip	201.5 KB
Uncompressed	1.3 MB
Compression rate	84.9 %
Selectors	23,792
ID Selectors	350
Class Selectors	27,488
Type Selectors	2,304
Attribute Selectors	2,098
Pseudo Classes	10,772
Pseudo Elements	4,218
Properties	184
Declarations	31,622
Rules	21,728
Vendor Prefixed	298
Property Resets	1,926
CSS Variables	459
Media Queries	46
Avg Rule Size	1.5 decl.
Max Rule Size	119 decl.
Avg Specificity	18.7
`!important`	50

---

5. Summary — Before / After

5.1. Recap table

Indicator	Before the session	After the session	Gain
Render-blocking CSS (gzip)	107.40 KB	58.97 KB	**−45 %**
CSS loaded on landing (cssstats)	367.9 KB	201.5 KB	**−45 %**
CSS selectors (cssstats)	41,330	23,792	**−42 %**
Unique CSS properties	241	184	**−24 %**
Lighthouse Performance Lab Score	(not measured)	98 / 100	—
PageSpeed Insights Performance	(with CSS blocker)	90 / 100	✅
LCP (DebugBear)	(before unblocking: > 2 s)	1.91 s	✅ Good
TBT	—	0 ms	✅ Perfect
CLS	—	0.02	✅ Good

5.2. Resulting architecture

Before:

```
main.jsx ──► xray.scss + custom + customizer + Swiper×6 + FA4 + Phosphor regular
             (EVERYTHING in the blocking entry bundle — 676 KB minified)
```

After:

```
main.jsx ──► xray-landing.scss + custom (light entry — 391 KB minified)
                          │
                          └──► non-blocking (preload + onload)

DefaultLayout (lazy) ──► xray-dashboard.scss + customizer + Swiper×6
                       + Phosphor regular/duotone/fill
                       (chunk fetched on first dashboard navigation — 496 KB)

deferred-icon-fonts.js ──► Remix Icon only (110 KB minified)
                           Still loaded async via index.html
```

---

6. Core Web Vitals compliance

With the final values:

Web Vital	Google "Good" target	Final value	Status
LCP	≤ 2.5 s	1.91 s (DebugBear) / 1.4 s (PSI)	✅ Good
FID / INP	≤ 200 ms	TBT 0–10 ms (proxy)	✅ Good
CLS	≤ 0.1	0.005–0.02	✅ Good

All Core Web Vitals are green. No Google ranking penalty. The landing is among the top ~5 % of the most performant websites.

---

7. Tools used

Tool	URL	Usage
Lighthouse (Chromium DevTools)	built-in	Local audit
PageSpeed Insights	`pagespeed.web.dev`	Official Google audit + opportunities
DebugBear	`debugbear.com/test`	Lighthouse Lab Score + filmstrip + Web Vitals
cssstats	`cssstats.com`	CSS volume / complexity, comparison to web average
Project Wallace	`projectwallace.com/analyze-css`	Structural CSS analysis, anomaly detection
Vite build report	`vite build`	Raw / gzip sizes for each JS and CSS chunk

---

8. Future improvements (optional)

If further optimization is desired later, sorted by decreasing gain and increasing risk:

1. **Inline critical CSS** (`vite-plugin-critical` or `critters`) — extract ~3 KB of CSS strictly required above the fold and inline it in `<head>` . Typical gain: 100–250 ms of FCP, which would push FCP below 1.8 s (the "Good" threshold).
2. **Icon-font subsetting** — the landing uses ~10 glyphs (`bi-telephone` , `bi-arrow-up` , `fa-map-marker-alt` , …). Replace the full FA5 + BI with a CSS+font subset, or with inline SVGs. Gain: ~25–35 KB additional gzip.
3. **PurgeCSS** on the landing bundle — static JSX analysis to remove Bootstrap classes never used on landing/auth. Gain: −40 to −80 KB. Risk: false negatives on dynamically-built classes.

4. `Speculation Rules` **API /** `<link rel="prefetch">` on the most likely routes (sign-in, about) to make navigation instantaneous.

At this stage (98 % Lab Score, all Core Web Vitals green), the effort/gain ratio becomes marginal for the landing. These tracks are worth considering only if a specific goal justifies them (e.g., reaching 100 / 100 on Lighthouse).

---

9. Appendix — Modified files

File	Type	Commit
`Front_End/CODE-REACT/vite.config.js`	modified	`a422e54`
`Front_End/CODE-REACT/src/main.jsx`	modified	`2f63a5d`
`Front_End/CODE-REACT/src/router/default-router.jsx`	modified	`2f63a5d`
`Front_End/CODE-REACT/src/layouts/defaultLayout.jsx`	modified	`2f63a5d` , `591aa72`
`Front_End/CODE-REACT/src/assets/scss/xray-landing.scss`	created	`2f63a5d`
`Front_End/CODE-REACT/src/assets/scss/xray-dashboard.scss`	created	`2f63a5d`
`Front_End/CODE-REACT/src/deferred-icon-fonts.js`	modified	`591aa72`

Total: 5 files modified, 2 files created, 3 commits.